

APIs Handoff

API List

API name	URL path	HTTP method	Purpose
Get all puzzles	<code>/api/getAllPuzzles</code>	GET	Return an object containing the three puzzles' IDs, titles, and summaries for selection.
Get one puzzle	<code>/api/getPuzzle/:puzzleId</code>	GET	Return the selected puzzle's opening story, current stage, question, clues, and progress; include its final reveal only after completion.
Submit an answer	<code>/api/submitAnswer/:puzzleId</code>	POST	Read <code>stageId</code> and a string <code>answer</code> , check the current stage, and return correct/wrong feedback and updated progress.
Request a stage hint	<code>/api/requestStageHint/:puzzleId</code>	PATCH	Read <code>stageId</code> , return the next hint's ID and text, and update hint usage for the current stage.

IDs and hint order

- `puzzleId` identifies the selected puzzle. `stageId` identifies a stage inside that puzzle; stage IDs repeat across puzzles.
- A hint's ID is its zero-based index in that stage's `hints` array: the first hint is `0`, the second is `1`.
- A hint ID is meaningful only with its puzzle and stage. Keep hint order fixed during play.
- The server chooses the next unused hint. The client sends the current stage ID, not a chosen hint ID.
- A hint response includes both `hintId` and `hint`, so the frontend can display the text without receiving unused hints.

Game flow and validation

- The list response has the shape `{ puzzles: [...] }`. It contains selection information, not complete stage data.
- The detail response provides the current stage's public content. Do not expose correct answers, unused hints, future-stage content, or the final reveal before completion.
- A wrong answer keeps the current stage unchanged and allows another attempt. A correct answer advances progress; the final correct answer marks the puzzle solved.
- The detail API provides the final reveal after completion. The final successful answer response may include it too; no separate result API is needed.

- Backend memory owns current stage, solved state, and hint usage. Validate input with plain checks.
- Return **400** with a clear message for missing, wrong-typed, or invalid fields. Return **404** for an unknown puzzle or stage.

Open Questions

- Agree on the exact detail, answer, and error response fields before implementation.
- Agree on how to report a stage that exists but is not current, an already solved puzzle, and exhausted hints. A **409** response is an option for these state conflicts; do not silently succeed or reveal more hints.
- Confirm one shared in-memory progress record per puzzle for this project's simple demo. No user accounts or authentication sessions.
- Confirm answer matching rules, including capitalization, surrounding spaces, and listed spelling variants.
- Eyad still needs to review and understand this list. His contribution and agreement must be recorded honestly before a paired submission.

Agreement and Contributions

- **Majed:** proposed the initial operations, accepted the list/detail division, approved identifying the puzzle in the URL and the stage in the request body, and chose hint-array indices as IDs.
- **Eyad:** review, contribution, and agreement are pending; no joint agreement is claimed.